

OpenMegatron: A Modular Research Software Platform for Ontology-Guided Memory and Skill-Routed LLM Agents

Yuan Zhe^a

^a*Independent Researcher, China*

Abstract

OpenMegatron is an open-source research software platform for constructing, running, and studying large language model agents with modular skills, structured memory, hybrid retrieval, and feedback-driven adaptation. The software combines a Python backend, a React/TypeScript user interface, versioned skill packs, a three-store memory subsystem using Redis, PostgreSQL/pgvector, and Neo4j, and optional trajectory-based reward modeling. Its goal is to provide researchers and practitioners with an inspectable agent workbench in which routing, memory, tool use, retrieval, and self-improvement components can be developed and evaluated as separable modules. This article describes the software architecture, core functionalities, example workflows, and research uses of OpenMegatron.

Keywords: LLM agents, research software, agent memory, retrieval augmented generation, skill routing, software engineering

Required Metadata

Current code version

1. Motivation and significance

Large language model (LLM) agents are increasingly used for programming assistance, literature review, data exploration, workflow automation, and interactive decision support. Unlike single-turn chat systems, agentic systems must coordinate multiple subsystems: model selection, tool invocation, skill discovery, memory storage, retrieval, error recovery, user confirmation, and

Nr.	Code metadata description	Please fill in this column
C1	Current code version	Preprint snapshot, git commit 1b61b70. Replace with the final release tag before submission.
C2	Permanent GitHub link to code/repository used for this code version	https://github.com/GodOn514/openMegatron . A Zenodo DOI should be minted for the submitted release.
C3	Legal Code License	MIT license declared in the repository README. A root LICENSE file should be added before submission.
C4	Code versioning system used	Git.
C5	Software code languages, tools, and services used	Python 3.11+, TypeScript, React, Vite, FastAPI, Redis, PostgreSQL, pgvector, Neo4j, SQLite, OpenAI-compatible model APIs, sentence-transformers, scikit-learn, PyTorch, Docker Compose.
C6	Compilation requirements, operating environments & dependencies	Windows, Linux, or macOS with Python 3.11+, Node.js 18+, and optionally Docker. Runtime services include Redis, PostgreSQL with pgvector, and Neo4j. Python dependencies are listed in <code>pysrc/requirements.txt</code> ; frontend dependencies are listed in <code>package.json</code> .
C7	If available Link to developer documentation/manual	Repository documentation: https://github.com/GodOn514/openMegatron . Local documentation includes README.md, README_CN.md, and CLAUDE.md.
C8	Support email for questions	yuan.zhe@example.com. Replace with the corresponding author’s production email before submission.

Table 1: Code metadata (mandatory).

logging. Research on generative agents, memory-augmented LLMs, retrieval-

augmented generation, tool learning, and open-ended agents has shown the potential of persistent agent state and structured tool use [1, 2, 3, 4, 5, 6]. However, many experimental agents are implemented as monolithic prototypes in which routing, memory, retrieval, and learning components are tightly coupled. This makes it difficult to reproduce results, compare design choices, or reuse individual modules in new research settings.

OpenMegatron addresses this gap by providing a modular research software platform for building and inspecting LLM agents. The software is organized around four design goals. First, agent behavior should be decomposed into explicit services: skill routing, model tier selection, memory, retrieval, caching, decision tracking, repair, and feedback collection. Second, memory should support more than flat vector retrieval. OpenMegatron therefore combines short-term conversation state, persistent episodic memory, graph-based relations, and an ontology that defines typed entities, relations, and hyperedge-like groupings. Third, skills should be versioned and discoverable. The software loads skills from directory-based packs containing metadata, instructions, and optional scripts. Fourth, agent improvement should be auditable. The platform records trajectories, feedback signals, reward-model features, and model registry entries so that adaptation can be studied without hiding the execution history.

The scientific problem solved by OpenMegatron is not a single benchmark task. Rather, it provides an experimental workbench for studying how LLM-agent subsystems interact in realistic multi-step workflows. Researchers can use it to examine questions such as: how should a task be routed to a skill pack; when should an agent use local memory, graph traversal, or hybrid RAG; what trajectory features predict successful execution; and how can error-repair strategies be evaluated across repeated tasks. These questions are relevant to software engineering, information retrieval, human–AI interaction, and knowledge management.

The intended experimental setting is a local or self-hosted workstation. A user starts the backend API and the React frontend, configures one or more OpenAI-compatible model providers, optionally starts Redis, PostgreSQL with pgvector, and Neo4j through Docker Compose, and interacts with the agent through the web interface or API. During a session, the agent routes requests to loaded skills, invokes tools, writes memory records, retrieves relevant context, and stores task traces. The same system can be run in a lightweight mode for skill and routing experiments or in a fuller mode with external storage and model APIs for memory and RAG experiments.

OpenMegatron is related to prior work in three areas. Retrieval-augmented

generation retrieves external information for generation, typically from vector stores [3]. GraphRAG-style systems enrich retrieval with entity and relation structure [7]. Memory-augmented agents maintain persistent state across interactions [1, 2]. OpenMegatron differs from these systems as a software contribution: it packages routing, tool execution, ontology-guided memory, RAG, feedback collection, and optional reward modeling into one inspectable platform, allowing researchers to modify and evaluate each component independently.

2. Software description

2.1. Software architecture

OpenMegatron uses a Python backend and a React/TypeScript frontend. The backend is the primary research software layer. It contains the core agent loop, memory engines, skill loader, routing logic, API endpoints, model dispatch, trajectory store, reward-model training utilities, and repair mechanisms. The frontend provides an operator interface for chat, runtime controls, skill editing, metrics, memory inspection, and related agent state.

At runtime, a user request enters the agent loop. The request is analyzed by the skill router, which scores candidate skills using metadata, keywords, loaded skill documentation, and task-intent signals. A model-tier component assigns the task to a lite, standard, or advanced model tier depending on complexity. The agent then composes context from short-term conversation state, persistent memory, RAG retrieval, and skill instructions. Tool calls are executed through a managed runtime and recorded with decision and trajectory metadata. Results are returned to the user and can be stored in memory for later retrieval.

The storage layer is intentionally heterogeneous. Redis stores short-term conversation buffers and cache entries. PostgreSQL with pgvector stores episodic memory records and vector embeddings. Neo4j stores graph-oriented relations and supports traversal use cases. SQLite is used for local trajectory and model registry persistence, which keeps learning experiments reproducible without requiring a separate analytics database. This separation makes the system suitable for controlled ablation experiments: researchers can run only the routing layer, add episodic memory, add graph memory, or add reward modeling as needed.

The ontology layer defines a typed memory schema. Its default schema includes memory records, entities, topics, sessions, projects, skills, tools,

claims, evidence, artifacts, owners, scopes, papers, authors, venues, literature reviews, decisions, alternatives, documents, communities, and memory cards. Relation types include mentions, topic, related, produces, uses, verified_by, belongs_to, cites, authored_by, published_in, reviews, surveys, extends, and member. Hyperedge types represent multi-party events such as memory capture, task experience, skill distillation, literature review, and decision recording. This schema is implemented in code rather than left as a narrative convention, which allows user interfaces, graph traversal, and memory export tools to share the same structure.

2.2. *Software functionalities*

OpenMegatron provides the following core functionalities.

1. **Skill loading and routing.** Skills are stored as versioned packs under category directories. Each pack can include a `SKILL.md` file, metadata, scripts, and supporting references. The router uses request features and skill descriptors to select an appropriate capability, while preserving a fallback route for unsupported tasks.
2. **Model-tier dispatch.** The model-tier subsystem separates low-cost, balanced, and advanced model classes. This allows experiments on cost-aware routing and permits the same agent workflow to run with different model providers.
3. **Ontology-guided memory.** The memory subsystem stores short-term and long-term context, links memories to entities and topics, and exposes graph-oriented recall functions. This is designed for multi-session tasks where prior decisions, artifacts, tools, and evidence should remain connected.
4. **Hybrid RAG.** The RAG layer supports local search through PostgreSQL with pgvector, global search through graph traversal, and fused search with reranking. Query classification routes factual, relational, and synthesis-oriented queries to different retrieval strategies.
5. **Decision tracking and repair.** Decisions can be logged with context, reasoning, and outcome. Operations can be wrapped with retry and recovery logic, making failures visible to later analysis rather than disappearing inside the agent loop.

6. **Trajectory collection and reward modeling.** Task traces can be stored as structured trajectories containing user input, selected skills, tool calls, reward, confidence, success, duration, and metadata. Feature extractors and reward scorers support offline experiments with scikit-learn or PyTorch models.
7. **API and user interface.** FastAPI endpoints and WebSocket event streams expose the backend to the React user interface. The interface supports conversation, skill inspection, memory-related workflows, runtime metrics, and operational controls.
8. **Research skill packs.** The repository includes skill categories for code engineering, research workflows, agent orchestration, media generation, office utilities, and monitoring. Research skills include paper reading, citation graph construction, evidence matrices, venue matching, citation verification, and top-paper search.

2.3. Implementation notes

OpenMegatron favors explicit, inspectable modules over hidden framework behavior. For example, SQL statements for local stores are declared as module-level constants, graph operations are wrapped in typed service classes, and trajectory features are extracted into fixed feature vectors. The test suite covers routing guardrails, ontology primitives, graph algorithms, cache behavior, repair hooks, reward-model training, auto-retraining logic, trajectory import/export, research utilities, and selected UI-adjacent workflows. In the inspected snapshot, 269 pytest test cases were collected, although one optional dependency used by the runtime setup tests must be installed before a full test run can complete.

The following simplified pseudocode shows the intended execution flow:

```
request = receive_user_message()
route = skill_router.match(request)
tier = model_tier.select(request, route)
context = memory.recall(request) + rag.retrieve(request)
result = repair_hook.execute(
    lambda: skill_runner.run(route.skill, request, context, tier)
)
decision_tracker.record(request, route, tier, result)
trajectory_store.store(result.trace)
memory.add_memory(session_id, result.summary, entities=result.entities)
```

3. Illustrative examples

This section describes representative workflows that can be reproduced with the repository after installing dependencies and configuring the model provider.

3.1. Starting a local research agent

The user installs Python and Node.js dependencies, starts the optional storage services, and launches the backend and frontend:

```
python scripts/runtime_setup.py
npm install
docker-compose up -d
python pysrc/agent.py --api
npm run dev
```

The web application then connects to the backend API. The user can ask the system to summarize a paper, search for relevant literature, inspect memory, or invoke a code-related skill. During the interaction, the agent selects a skill pack and records operational metadata. This example demonstrates how OpenMegatron can be used as a local, inspectable alternative to hosted agent systems.

3.2. Ontology-guided memory and retrieval

A researcher studying long-term agent memory can create a session, store task events, and query previous decisions. The memory service writes short-term messages to Redis, inserts persistent episodic records into PostgreSQL, and mirrors relations into the graph layer when available. The ontology provides common node and relation labels, allowing the same records to be inspected as text, vector-searchable memory, or graph-connected evidence. This supports experiments comparing flat retrieval with relation-aware retrieval without changing the surrounding agent loop.

3.3. Trajectory-based analysis

A second user may focus on agent reliability. After executing a set of tasks, the trajectory store can be queried by session, source, success flag, date range, or offset. Reward-model features include tool count, duration, selected skill count, error-tool ratio, verification signals, feedback labels, and source indicators. These features can be used to train lightweight reward scorers and evaluate whether new scoring models improve over prior versions. This

workflow is useful for studying agent improvement as an engineering process rather than as an opaque update.

3.4. Research workflow skills

OpenMegatron also includes research-oriented skills. A literature-review workflow can search papers, read abstracts, build citation relations, verify citation metadata, and construct evidence matrices. The same skill system can host code-assistant, code-review, code-test, office, and media skills. The common interface makes it possible to evaluate how routing behavior changes as the skill market grows.

4. Impact

OpenMegatron’s main impact is to provide a reusable testbed for LLM-agent systems research. Instead of building isolated prototypes for memory, routing, retrieval, and adaptation, researchers can use one software platform with separable modules and shared logs. This enables new research questions in at least four directions.

First, OpenMegatron supports research on structured agent memory. The ontology and graph components make it possible to compare short-term memory, vector retrieval, graph traversal, and hybrid recall in the same environment. This is useful for studying whether relation-aware memory improves multi-session tasks, project continuity, or literature-review workflows.

Second, the platform supports research on skill routing and tool composition. Because skills are versioned directory packs with explicit metadata, researchers can evaluate how descriptors, keywords, category priors, and model-tier decisions affect routing accuracy and execution cost. This is relevant to tool-use research and to practical deployment of agent workbenches.

Third, OpenMegatron supports reliability and feedback studies. Trajectory records preserve state, actions, rewards, confidence, tool usage, and verification signals. This allows researchers to analyze which tool sequences fail, which error patterns are recoverable, and which feedback labels are most predictive of successful tasks. The reward-model components are intentionally lightweight so that the same traces can be reused in small local experiments.

Fourth, the software can improve daily research practice for individual users. A researcher can use one local system for paper search, citation verification, code assistance, RAG ingestion, and memory-backed project continuation.

Because the system is self-hosted and inspectable, users can audit what was stored, which skill was selected, and which tool calls were executed.

At the time of writing, OpenMegatron should be viewed as an active research software artifact rather than a mature production service. Its repository contains the primary code, documentation, scripts, and tests needed for continued development. Before final SoftwareX submission, the project should provide a stable release tag, a permanent archive DOI, a root license file, a passing dependency-locked test run, and a short demonstration video. These additions would make the software easier to review, cite, and reuse.

5. Conclusions

OpenMegatron is a modular research software platform for studying LLM agents that combine skill routing, ontology-guided memory, hybrid retrieval, tool execution, decision tracking, and feedback-driven adaptation. Its architecture separates the main agent loop from memory, graph, RAG, routing, repair, API, and trajectory components, making it suitable for controlled software experiments and practical agent prototyping. The platform is particularly useful for researchers interested in long-term agent memory, skill ecosystems, agent reliability, and reproducible tool-using workflows. By packaging these capabilities into an open-source codebase with a web interface and local persistence layers, OpenMegatron offers an extensible foundation for future studies of modular LLM-agent systems.

Conflict of Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.

Acknowledgements

The author thanks the open-source communities behind Python, FastAPI, React, Redis, PostgreSQL, pgvector, Neo4j, sentence-transformers, scikit-learn, PyTorch, and related tools used by OpenMegatron.

Executable software metadata	Value
Software name	OpenMegatron
Developers	Yuan Zhe and contributors
Repository	https://github.com/GodOn514/openMegatron
Operating system	Windows, Linux, macOS
Programming languages	Python, TypeScript
External services	Optional Redis, PostgreSQL with pgvector, Neo4j, and OpenAI-compatible model APIs
Installation	<code>python scripts/runtime_setup.py</code> ; <code>npm install</code> ; optional <code>docker-compose up -d</code>
Execution	<code>python pysrc/agent.py -api</code> ; <code>npm run dev</code> ; or <code>start.bat</code> on Windows

Table 2: Current executable software version.

Current executable software version

References

- [1] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein, Generative agents: Interactive simulacra of human behavior, in: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST), 2023. doi:10.1145/3586183.3606763. URL <https://doi.org/10.1145/3586183.3606763>
- [2] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, J. E. Gonzalez, MemGPT: Towards LLMs as operating systems, arXiv preprint (2024). arXiv:2310.08560. URL <https://arxiv.org/abs/2310.08560>
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems 33 (NeurIPS), 2020. doi:10.5555/3495724.3495725. URL <https://doi.org/10.5555/3495724.3495725>
- [4] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing

- Systems 36 (NeurIPS), 2023. [arXiv:2302.04761](#).
URL <https://arxiv.org/abs/2302.04761>
- [5] S. G. Patil, T. Zhang, X. Wang, J. E. Gonzalez, Gorilla: Large language model connected with massive APIs, in: Advances in Neural Information Processing Systems 36 (NeurIPS), 2023. [arXiv:2305.15334](#).
URL <https://arxiv.org/abs/2305.15334>
- [6] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, A. Anandkumar, Voyager: An open-ended embodied agent with large language models, in: Transactions on Machine Learning Research (TMLR), 2024. [arXiv:2305.16291](#).
URL <https://arxiv.org/abs/2305.16291>
- [7] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, J. Larson, From local to global: A graph RAG approach to query-focused summarization, arXiv preprint (2024). [arXiv:2404.16130](#).
URL <https://arxiv.org/abs/2404.16130>